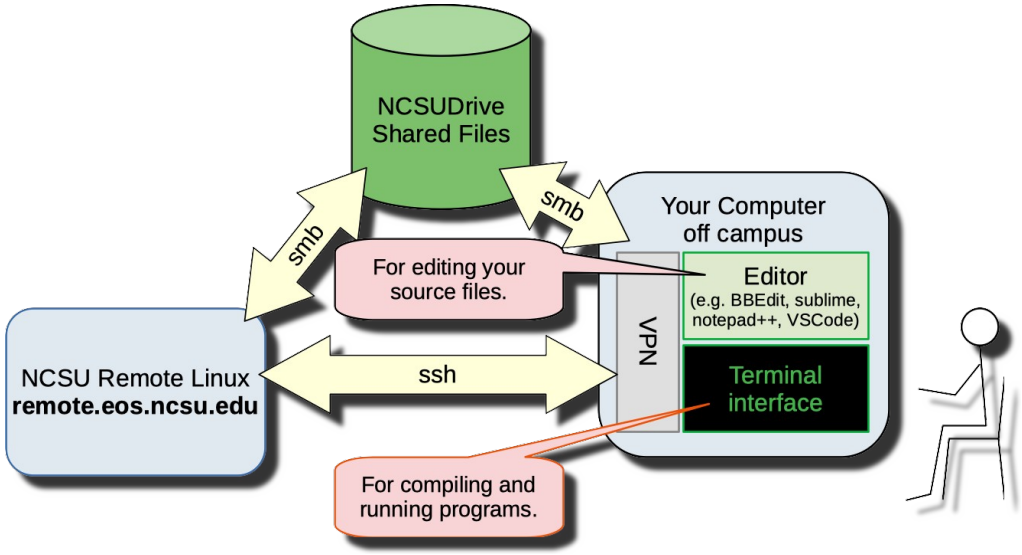
How I Completed Exercise 01
From My Macintosh using ssh and NCSU Drive

For this example, I kept my files in my NCSU Drive space. This made all the files easy to access from either my own laptop or from an NCSU Linux machine. I ran an editor on my laptop and used one of the NCSU Remote Linux machines to compile and run my program.



Preparing your Environment

You'll need to install and set up some software before you can work on the exercise this way.

VPN to NC State

To access your NCSU Drive files from off campus, you'll need to set up a Virtual Private Network (VPN) connection to the NCSU Campus. This will make your system look like it's part of the campus network, even if you're connecting from elsewhere. An NCSU VPN connection will also let you access some other resources you'll need later, like our Jenkins system used for building and testing your projects.

The university has instructions for installing, configuring and connecting with the VPN software. You may have already followed this procedure for one of your earlier classes:

https://oit.ncsu.edu/campus-it/campus-data-network/vpn/

Mounting your NCSU Drive

NC State offers shared file space to students through NCSU Drive. This will let you access the same files from your own laptop or desktop and from university systems. This can be particularly useful in CSC 230, where you may want to edit files from your own machine and then compile, run and debug them from a university machine with the same software configuration we'll use when we're testing your code.

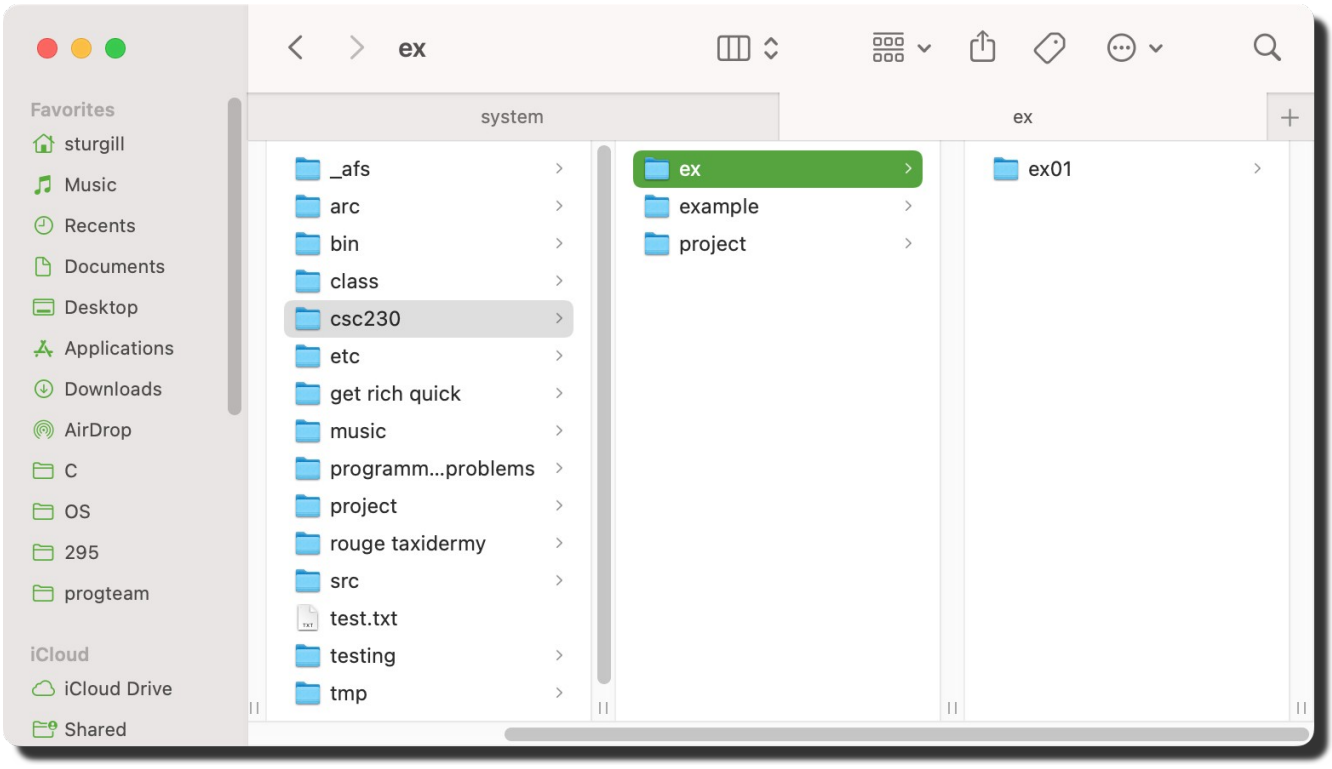
Once you have connected to campus with VPN, accessing your NCSU Drive space is easy. On a MacOS system, you should be able to:

- Open Finder (the file browser program)
- From the menu, select **Go → Connect to Server…**
- Enter the following path in the combo box at the top. You may want to click the plus button to save this as one of your favorites.
  smb://ncsudrive.ncsu.edu/home
- Press **Connect** and authenticate with your NCSU Credentials. This should bring up a file browser window for your home NCSU Drive space.
- After connecting to this share, I was able to access it through the finder, or from the terminal window, it showed up under the following path. This made it easy to access my files either through the GUI or from the command line if I needed to.
  `/Volumes/home`

The Office of Information Technology provides more detailed instructions for accessing and using your NCSU Drive from a MacOS System:

https://ncsu.service-now.com/sp?id=kb_article_view&sysparm_article=KB0018178

In my NCSU Drive space, I made a directory for working on my CSC 230 assignments, and, under that, I made a directory for exercises, with a new directory for exercise 1 (csc230/ex/ex01). You can organize your files however you want, but it might be convenient to keep all your CSC 230 work in one place.



Choosing an Editor

You will want a good text editor for writing your C programs and editing other text files. The following options have been popular on MacOS. They all know how to syntax-highlight code in C and lots of other languages.

- BBEdit
  Available in the App Store
- Sublime
  Available from: https://www.sublimetext.com
- VSCode
  Available from: https://code.visualstudio.com

You may have already installed one of these or some other editor you like in a previous CSC class.

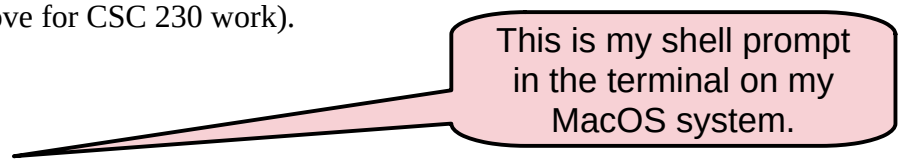
Remote Terminal Connection

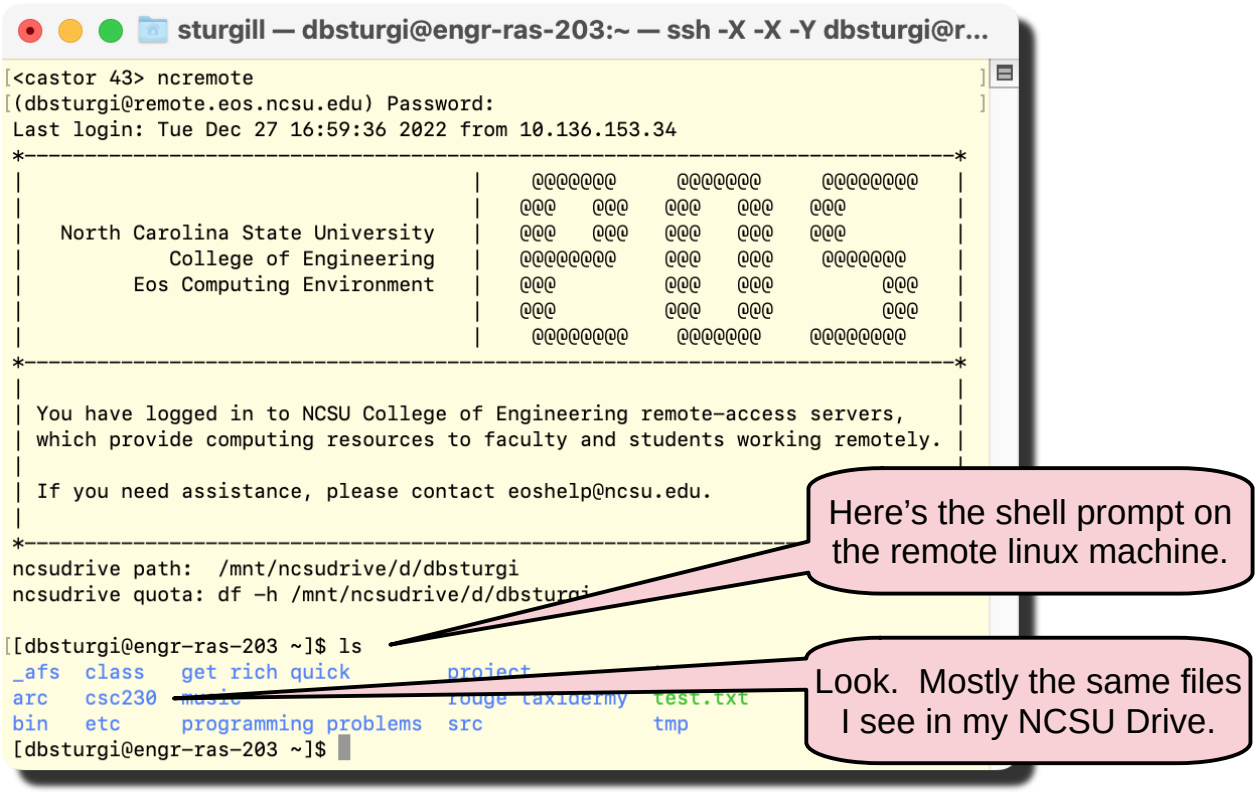
MacOS should already have a secure shell (ssh) client that will let you log in to one of the NCSU remote Linux machines. You can run it from the terminal program. I opened a terminal window from Application → Utilities → Terminal. In my terminal window, I entered the following command to log in remotely to a university Linux machine. Replace the ***unity_id*** with your actual NCSU Unity ID, and don't actually type the dollar sign at the start, that's not part of the command.

```
$ ssh unity_id@remote.eos.ncsu.edu
```

After authenticating, I got a shell prompt on the remote Linux system. In the terminal window, your *shell* is the command interpreter; it runs the commands you enter. In showing shell commands (like the one above), the $ at the start of each line stands for a shell prompt. It indicates where you should enter a shell command. Don't really type the dollar sign in these example commands.

Entering the **ls** command at the shell prompt let me see the same files in my NCSU Drive (including the csc230 directory I made above for CSC 230 work).





Downloading the Starter Files

On my MacOS system, I downloaded the starter files from the course Moodle page using my web browser. From the Moodle page, I clicked on Exercise 1 Files then downloaded each of the two files (**arithmetic.c** and **expected.txt**) by right-clicking and then selecting **Save Linked Content As...** This let me choose to save each file under the ex01 directory I made above, and it helped me make sure I wouldn't accidentally miss some of the file's content or modify a file in the process of, say, copy-and-pasting it from web browser. In general, you don't want to download files for this class via copy-and-paste.

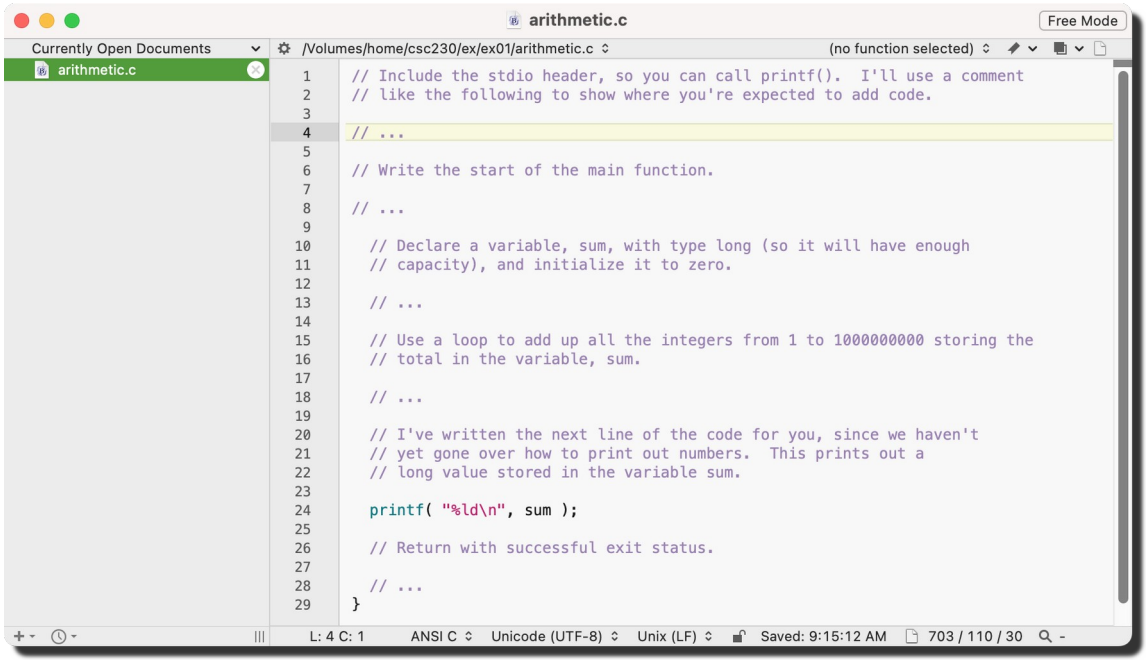
Instead of getting the starter files them from the Moodle page, I could have copied them from my ssh login to the university remote Linux machine. Like it says in the exercise 01 write-up, I could have changed to my work directory for exercise 01, then copied the starter files with commands like:

```
$ cd csc230/ex/ex01
$ cp /mnt/coe/workspace/csc/CSC230/ex/exercise01/* .
$ ls
arithmetic.c  expected.txt
```

Don't forget the dot at the end.

Completing the Program

Once you have your own copy of the starter files, you can start the text editor of your choice and use it to modify the arithmetic.c source file. In your editor, open the copy of arithmetic.c that you downloaded to your NCSU drive. Here's what it looked like when I opened the source file in BBEdit:



In the editor, I added the code I needed to and then saved the file. I kept my editor window running, in case I needed to fix anything later.

Building and Testing

Then, back in my ssh login window, I compiled and ran the program. Since I'm working in my NCSU Drive space, saving changes from my editor should automatically update the same files I'm seeing from the NCSU Linux machine.

The commands below compile the program with our usual compiler options. Then, I run the program, using the **tee** program to send its output to the screen and to a file named output.txt. This makes it easy to see the output right away and then to check it later against the expected output to make sure it's exactly right. After running my new program, I check the exit status of my program to make sure it exited successfully. Finally, I use the **diff** program to make sure my program's output (saved to the file named output.txt) looks exactly like the expected output.

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
$ ./arithmetic | tee output.txt
5000000000500000000
$ echo $?
0
$ diff output.txt expected.txt
```

You should plan to check your output against the expected output on every exercises. That's what the grading program will do when we're evaluating your work. The automatic grader isn't smart enough to tell the difference between important and unimportant differences in your program's output, so be sure to use diff to make sure your output is exactly right. Don't depend on just looking at your programs output in the terminal window. This could make it easy to miss a small difference that will cost you points when the automated grader looks at your output.

In the sample execution above, everything looks good. The diff program didn't find any differences between my output and the expected output. And, since I compiled and executed the program on an EOS Linux machine, my program should produce the same results when it's graded.

Submitting the Solution

Once I was happy with my solution, I submitted my **arithmetic.c** source file for the exercise_01 assignment on Moodle. My NCSU Drive home directory was still connected, so, when I went to add files to my submission for the assignment, I was able to choose the **arithmetic.c** file from the NCSU Drive and submit that.